

CCAR-F

Claude Certified Architect - Foundations 실전 학습 가이드

출력해서 밑줄 긋고, 체크하고, 직접 설계해보는 형태
2026-09 기준 | 개인 학습용

이 자료의 목적

시험 문제를 암기하는 것이 아니라, CCAR-F 가 요구하는 “설계 판단”을 빠르게 익히는 것입니다. 각 장은 개념 → 판단 기준 → 흔한 오답 → 손으로 푸는 연습 순서로 구성했습니다.

권장 사용법: 하루 45~60 분 × 8~10 회 + 모의문제 1~2 회

※ 본 문서는 Anthropic 공식 시험문제가 아니며, 공개된 시험 범위와 공식 제품 문서를 바탕으로 만든 독립 학습자료입니다.

1. 시험을 먼저 이해하기

시험 개요

항목	내용
시험	Claude Certified Architect - Foundations (CCAR-F)
대상	Claude 기반 production solution 을 설계하는 architect / technical practitioner
문항/시간	60 문항 / 120 분
언어	영어
형태	시나리오 기반 multiple choice / multiple response
합격점	Scaled score 720 / 1000
유효기간	12 개월
핵심	"아는가?"보다 "이 상황에서 어떤 구조가 더 적절한가?"

도메인 비중

도메인	비중	학습 우선순위
Agentic Architecture & Orchestration	27%	최우선
Tool Design & MCP Integration	18%	높음
Claude Code Configuration & Workflows	20%	높음
Prompt Engineering & Structured Output	20%	높음
Context Management & Reliability	15%	중요

시험의 중심 사고

정답은 "가장 화려한 아키텍처"가 아니라, 요구사항을 충족하면서 복잡성·권한·비용·실패면적을 최소화하는 설계인 경우가 많습니다.

재형님에게 유리한 점

- Python 으로 실제 AI/최적화 시스템을 만든 경험이 있어 "구현 가능한 설계" 감각이 이미 있습니다.
- OptiQube 처럼 solver/tool 기반 Agent 구조를 고민하고 있어 Domain 1·2 를 실제 프로젝트와 연결할 수 있습니다.
- 반대로 Claude Code 고유 설정(CLAUDE.md, settings, hooks, skills)과 Anthropic 식 용어는 시험 대비로 별도 정리가 필요합니다.

2. CCAR-F 에서 반복되는 판단 원칙

1) 가장 단순한 구조부터

한 번의 모델 호출로 충분하면 agent 를 만들지 않습니다. 하나의 agent 로 충분하면 multi-agent 를 만들지 않습니다.

2) 독립적인 일만 병렬화

서로 결과를 기다릴 필요가 없는 작업은 parallel. 선행 결과가 다음 입력을 결정하면 sequential.

3) Context 는 공유 자원이자 비용

모든 것을 한 context 에 넣기보다, 필요한 정보만 해당 agent/tool 에 전달합니다.

4) 권한은 최소화

subagent 와 tool 에 필요한 최소 권한만 부여합니다. 특히 쓰기/삭제/배포 같은 비가역 행동은 명확한 통제 지점을 둡니다.

5) 모델에게 암묵적으로 기대하지 않기

출력 기준, tool 용도, 성공/실패 조건을 명시적으로 적습니다.

6) 실패를 정상 상태로 설계

timeout, partial failure, malformed result, ambiguous input 이 “예외”가 아니라 정상적으로 발생할 수 있는 상태라고 봅니다.

7) 구조화된 경계

agent ↔ tool, model ↔ application 경계에서는 schema/typed output 이 reliability 를 크게 올립니다.

8) Human-in-the-loop 은 필요한 곳에만

모든 step 을 승인받으면 자동화 가치가 사라지고, 아무 승인도 없으면 위험이 커집니다. 영향도가 큰 행동 전에 배치합니다.

빠른 선택 공식

상황	먼저 떠올릴 것
툴을 자꾸 잘못 선택한다	router 추가 전에 tool name/description/schema 를 먼저 개선
출력 형식이 흔들린다	자유 텍스트 후 regex 보다 structured output/schema
긴 작업 중 맥락이 무너진다	summary/checkpoint/session state 분리
독립 조사 4 개가 느리다	subagent 병렬 실행 고려
한 단계 결과로 다음 단계가 달라진다	sequential workflow
외부 시스템 삭제/배포	permission + human approval + audit
agent 가 과도하게 일을 벌인다	scope/stop condition/max turns/explicit criteria
하위 agent 가 너무 많은 정보를 본다	task-specific context + least privilege

3. Domain 1 - Agentic Architecture & Orchestration (27%)

이 장의 목표

agentic loop, coordinator-subagent, 병렬/순차 orchestration, hooks, session state 를 보고 “왜 이 구조를 쓰는지” 설명할 수 있어야 합니다.

3.1 Agentic loop

기본 흐름은 대략 다음처럼 생각하면 됩니다: 요청 이해 → 계획/판단 → tool/action → 결과 관찰 → 다음 행동 결정 → 종료. 중요한 것은 “반복” 자체가 아니라 종료 조건과 실패 처리입니다.

- ☐ 명확한 stop condition 이 있는가?
- ☐ 최대 turn / 시간 / 비용 제한이 필요한가?
- ☐ tool result 가 실패했을 때 retry 할지, 다른 전략으로 바꿀지, human 에게 넘길지 정했는가?
- ☐ 비가역 행동 전에 통제 지점이 있는가?

3.2 Single agent vs multi-agent

선택	적합한 상황	주의점
Single agent	상태 공유가 중요하고 단계 간 의존성이 큼	context 가 비대해지기 쉬움
Coordinator + subagents	독립 workstream, 전문 역할, 격리된 context 가 이점	handoff·merge·failure 관리 필요
고정 workflow	절차와 순서가 명확하고 예측 가능	유연성은 낮지만 reliability 높음

3.3 병렬 vs 순차

- Parallel: 서로 독립이며 결과를 합치기만 하면 되는 조사/검사/분석.
- Sequential: 앞 단계의 결과가 다음 행동을 결정하거나, 동일 리소스를 순서대로 수정해야 하는 경우.
- Hybrid: 병렬 조사 → coordinator merge → 순차 실행처럼 혼합하는 경우가 실무에서 흔합니다.

3.4 Hooks

Hook 은 모델의 “판단”에 맡기지 않고 프로그램적으로 정책을 강제할 때 유용합니다. 예: 특정 tool 호출 전 입력 정규화, 위험 명령 차단, 감사 로그 기록, 승인 여부 판정.

시험 포인트

“프롬프트에 하지 말라고 써두기”와 “프로그램적으로 막기” 중 후자가 필요한 상황을 구분하세요. 보안/정책/비가역 행동은 deterministic enforcement 가 더 적절합니다.

3.5 Session / Resume / Fork

- Resume: 같은 작업의 상태와 맥락을 이어서 계속해야 할 때
- Fork: 기존 상태를 보존하면서 다른 가설/분기를 독립적으로 탐색할 때
- Checkpoint: 긴 작업에서 중간 산출물을 저장해 실패 시 전체 재실행을 피할 때

OptiQube Lab A

아래를 종이에 직접 그려보세요.

- ☐ 사용자 자연어 요청 → 문제 정의 agent → 데이터 검증 → solver 호출 → 결과 해석 → 최종 답변 흐름
- ☐ 데이터 검증과 참고자료 조회를 병렬로 돌릴 수 있는지 표시
- ☐ solver infeasible 일 때 retry 가 아니라 “diagnosis branch”로 보내는 조건 정의
- ☐ solution export / DB write 같은 행동에 human approval 이 필요한지 표시

4. Domain 2 - Tool Design & MCP Integration (18%)

4.1 좋은 tool interface 의 조건

- ☐ 이름이 행동을 명확히 표현한다.
- ☐ description 에 “언제 사용해야 하는지 / 언제 사용하면 안 되는지”가 드러난다.
- ☐ input schema 가 모호하지 않고 필수/선택 필드를 구분한다.
- ☐ 결과가 parsing 가능한 구조를 가진다.
- ☐ 실패도 structured error 로 반환한다.
- ☐ tool 하나가 너무 많은 일을 하지 않는다.

4.2 Tool description 이 중요한 이유

LLM 에게 tool description 은 사실상 API documentation 이자 routing signal 입니다. 잘못된 tool 선택 문제가 생겼을 때 별도 classifier 를 추가하기 전에 description 과 schema 를 먼저 개선하는 것이 보통 더 단순하고 효과적입니다.

4.3 Structured error

나쁜 예

"Error occurred"

좋은 방향

error_type, retryable, message, invalid_fields, suggested_action 처럼 application 과 agent 가 다음 행동을 결정할 수 있는 정보를 반환합니다.

4.4 MCP mental model

MCP 는 AI application 이 외부 data/tool 에 연결되는 방식을 표준화하는 protocol 입니다. 시험에서는 protocol 세부 wire format 보다 “어떤 기능을 MCP server 로 제공할지, 권한/인증을 어떻게 제한할지, tool surface 를 어떻게 설계할지” 판단하는 감각이 중요합니다.

- 사내 DB / Jira / GitHub / solver 같은 외부 capability 를 표준화된 interface 로 노출
- tool 수가 지나치게 많으면 선택 혼란이 커질 수 있으므로 목적별로 정리
- 인증과 권한은 MCP 연결 자체와 별개로 명확히 설계
- 민감 데이터는 필요한 agent/tool 에만 노출

4.5 Built-in tool vs custom tool

이미 플랫폼이 제공하는 built-in capability 로 충분하면 custom tool 을 만들 이유가 없습니다. 반대로 비즈니스 규칙, 사내 시스템, 특수 입력 검증이 필요하다면 custom tool/MCP 가 적절합니다.

OptiQube Lab B - solver tool schema

- ☐ solve_optimization(problem_id, objective, constraints, options) 형태의 입력 schema 초안
- ☐ result 에 status / objective_value / solution / warnings / diagnostics 를 분리
- ☐ INFEASIBLE / TIMEOUT / INVALID_INPUT 을 서로 다른 error/status 로 표현
- ☐ DB write/export tool 은 read-only analysis tool 과 분리

5. Domain 3 - Claude Code Configuration & Workflows (20%)

이 영역은 재형님이 별도 암기해야 할 가능성이 큼니다.

개발 실력보다는 Claude Code 가 설정을 어떻게 상속하고, 프로젝트 지침/권한/작업 방식을 어디에 두는지 묻는 문제에 대비해야 합니다.

5.1 CLAUDE.md

CLAUDE.md 는 코드베이스의 "작업 지침서"처럼 생각하면 좋습니다. 프로젝트 구조, 테스트 명령, 코딩 규칙, 중요한 제약을 적어 Claude Code 가 반복적으로 참고하도록 합니다.

- 짧고 구체적인 규칙
- 실행 가능한 명령 예시
- repo 마다 달라지는 conventions
- 절대 해서는 안 되는 위험 행동
- 불필요한 장문 설명보다 작업에 직접 도움이 되는 지침

5.2 Settings 우선순위

시험에서는 enterprise / CLI / local project / shared project / user 설정이 충돌할 때 무엇이 우선하는지 판단하는 문제가 나올 수 있습니다. 핵심은 조직 정책이 개인 설정보다 강하다는 점입니다.

기억용

Enterprise managed policy > command-line > project local > project shared > user settings

5.3 Plan mode

바로 파일을 수정하기보다 먼저 탐색·계획만 하도록 제한하고 검토한 뒤 실행하는 패턴입니다. 큰 변경, 익숙하지 않은 repo, 위험도가 높은 작업에서 특히 유용합니다.

5.4 Skills / Commands / Hooks / Subagents 구분

개념	역할
CLAUDE.md	지속적인 프로젝트 지침/맥락
Skill	재사용 가능한 전문 작업 지식/절차
Command	사용자가 명시적으로 호출하는 반복 작업 shortcut
Hook	특정 lifecycle/tool event 에서 프로그램적으로 실행/검사
Subagent	격리된 context 와 역할로 위임되는 별도 agent

5.5 CI/CD 와 non-interactive 실행

자동화에서는 interactive prompt 에 의존하면 안 됩니다. 출력 형식, 최대 turn, 권한, 실패 code, 로그 등을 deterministic 하게 다루는 것이 핵심입니다.

손으로 확인

- ☐ CLAUDE.md 에 넣을 것과 hook 으로 강제할 것을 구분할 수 있다.
- ☐ 개인 설정과 팀 공유 설정을 구분할 수 있다.
- ☐ plan mode 가 필요한 상황을 설명할 수 있다.
- ☐ subagent 와 skill 의 차이를 설명할 수 있다.

6. Domain 4 - Prompt Engineering & Structured Output (20%)

6.1 Clear & explicit

Claude 는 명확하고 직접적인 지시에 잘 반응합니다. 목표, 제약, 성공 기준, 출력 형식이 구체적일수록 안정적입니다.

6.2 Few-shot

말로 규칙을 길게 설명하는 것보다, 대표적인 좋은 입출력 예시 몇 개가 효과적인 경우가 많습니다. 단 예시는 모델이 그대로 일반화하므로 잘못된 습관이 섞이지 않게 해야 합니다.

6.3 Structured output

- ☐ application 이 결과를 파싱해야 한다
- ☐ DB/API 로 후속 처리를 한다
- ☐ 필수 field 누락이 치명적이다
- ☐ enum/숫자/배열 타입을 보장해야 한다

위 조건이 많을수록 자유 텍스트보다 schema 기반 structured output 이 적합합니다.

6.4 Validation + retry

출력 검증 실패 시 무조건 동일 요청을 반복하기보다, 실패 원인을 모델에 다시 제공하거나 schema/criteria 를 보강해 targeted retry 를 하는 편이 낫습니다. 반복 횟수는 제한합니다.

6.5 Multi-pass review

생성과 검토를 분리하면 품질이 올라갈 수 있지만, 모든 작업에 reviewer agent 를 추가하는 것은 과도한 복잡성입니다. 오류 비용이 높거나 기준이 명확한 경우에 사용합니다.

Prompt 개선 순서

1. 요구사항/성공기준을 더 명확히
2. 출력 형식/schema 지정
3. 좋은 예시 추가
4. 작업 분해
5. 검증/재시도
6. 필요할 때만 별도 reviewer/agent

시험 합정

처음부터 “다른 모델 하나를 더 붙여 검증”하는 답보다, 명확한 criteria / schema / tool description 처럼 더 작은 수정이 우선인 경우가 많습니다.

7. Domain 5 - Context Management & Reliability (15%)

7.1 Context window 는 무한 메모리가 아니다

긴 대화와 대규모 repo 에서는 중요 정보가 묻히고 비용·latency 가 증가합니다. 모든 것을 계속 누적하지 말고, task state 와 대화 history 를 구분해서 관리합니다.

7.2 Context 전략

Selective context: 현재 task 에 필요한 정보만 넣기

Summarization: 오래된 history 를 핵심 결정/미해결사항 중심으로 압축

Checkpoint: 긴 workflow 에서 중간 결과를 외부 state 로 저장

Prompt caching: 반복되는 큰 prefix 를 재사용해 cost/latency 감소

Subagent isolation: 독립 작업은 별도 context 로 분리

7.3 Ambiguity 와 escalation

모호한 입력을 임의 추정해서 장시간 실행하는 것보다, 결과에 큰 영향을 주는 핵심 ambiguity 는 실행 전에 clarification 을 받는 편이 낫습니다. 반대로 영향이 작고 되돌릴 수 있는 항목까지 모두 물으면 자동화 가치가 떨어집니다.

7.4 Reliability checklist

- ☐ retry 가능한 오류와 불가능한 오류를 구분
- ☐ 중복 실행(idempotency) 고려
- ☐ 부분 성공(partial success)을 표현
- ☐ provenance / audit trail 보존
- ☐ human review 가 필요한 risk boundary 정의
- ☐ timeout / max turn / budget limit 설정

OptiQube Lab C - context 설계

- ☐ 사용자 원문, 정규화된 optimization spec, solver state, 결과 요약을 서로 분리
- ☐ 대규모 원본 데이터는 context 에 통째로 넣지 않고 tool 로 조회
- ☐ 이전 solver run 의 핵심 decision 만 session state 에 보존
- ☐ 사용자가 목표함수를 바꾸면 무엇을 재사용하고 무엇을 폐기할지 결정

8. 시나리오 문제 푸는 순서

7. 먼저 “문제가 무엇인지” 한 문장으로 재정의한다. (tool selection? context? permission? output reliability?)
8. 가장 작은 수정으로 해결 가능한지 본다.
9. 선택지 중 새로운 component 를 추가하는 답은 필요성이 명확한지 의심한다.
10. 권한과 failure blast radius 를 본다.
11. 병렬/순차 의존성을 확인한다.
12. 모델에게 맡겨야 할 판단과 코드로 강제해야 할 정책을 구분한다.
13. 비용/latency/운영 복잡성까지 포함해 최종 선택한다.

자주 보게 될 대비

A	B
Prompt instruction	Hook / deterministic enforcement
Single agent	Coordinator + subagent
Sequential	Parallel
Free-form output	Structured output
Full history	Summarized/selective context
Custom router	Better tool description
Auto execution	Human approval boundary
Retry everything	Typed error + targeted recovery

9. 연습문제 - 먼저 답을 표시하세요

실제 시험문제가 아닌 독립 제작 문제입니다. 각 문항에서 가장 설계적으로 적절한 답을 고르세요. 정답/해설은 다음 장에 있습니다.

Q1. 한 agent 가 서로 독립적인 5 개 저장소를 조사한 뒤 공통 패턴만 합쳐야 한다. 가장 적절한 구조는?

- A. 한 agent 가 저장소를 하나씩 순서대로 조사
- B. 각 저장소를 독립 subagent 가 병렬 조사하고 coordinator 가 merge
- C. 모든 파일을 한 번에 context 에 넣기
- D. 5 개의 tool router 를 계층적으로 추가

내 답: _____ 확신도: 1 2 3 4 5

Q2. Claude 가 weather tool 대신 location lookup tool 을 자주 선택한다. 두 tool 기능이 겹치지 않는다. 우선 조치는?

- A. 별도 classifier model 추가
- B. tool name/description/schema 를 더 명확히
- C. temperature 를 낮춤
- D. 모든 tool 을 하나로 합침

내 답: _____ 확신도: 1 2 3 4 5

Q3. 배포 명령은 실수 시 고객 서비스에 영향을 준다. 어떤 통제가 가장 적절한가?

- A. system prompt 에 조심하라고 적기
- B. 삭제/배포 tool 을 완전 제거
- C. 실행 직전 deterministic policy/human approval boundary
- D. 매번 reviewer agent 를 3 개 호출

내 답: _____ 확신도: 1 2 3 4 5

Q4. 3 단계 workflow 에서 2 단계 입력은 1 단계 결과에 따라 달라진다.

- A. 3 단계 모두 병렬
- B. 순차 실행
- C. 랜덤 순서
- D. 각 단계를 별도 context 없이 반복

내 답: _____ 확신도: 1 2 3 4 5

Q5. application 이 Claude 출력의 status, confidence, ids 를 후속 API 로 전달해야 한다.

- A. 자유 텍스트 + regex
- B. structured output/schema
- C. markdown table
- D. temperature 0 만 사용

내 답: _____ 확신도: 1 2 3 4 5

Q6. 긴 분석 session 에서 이전 대화가 너무 길어 비용과 집중력이 악화된다.

- A. 매 turn 전체 history 복제
- B. 핵심 결정/미해결사항을 요약하고 task state 를 분리
- C. model 을 더 큰 것으로 변경
- D. 모든 history 를 subagent 에 복제

내 답: _____ 확신도: 1 2 3 4 5

Q7. 회사 전체에서 절대 실행하면 안 되는 shell command 를 막아야 한다.

- A. CLAUDE.md 에 금지 문구만 추가
- B. hook/managed policy 로 프로그램적으로 차단
- C. 사용자에게 기억하라고 교육
- D. 출력 뒤에 검사

내 답: _____ 확신도: 1 2 3 4 5

Q8. 하위 agent 가 고객 DB 를 읽기만 하면 되는 분석을 수행한다.

- A. DB admin 권한 부여
- B. 필요한 read-only tool 만 제공
- C. 전체 MCP server 의 모든 tool 제공
- D. coordinator 의 자격증명을 그대로 공유

내 답: _____ 확신도: 1 2 3 4 5

Q9. 한 tool 이 VALIDATION_ERROR 와 TEMPORARY_TIMEOUT 을 모두 문자열 "failed"로 반환한다.

- A. 좋은 설계
- B. error type 과 retryability 를 구조화해야 함
- C. 모델 크기를 키우면 해결
- D. tool 을 삭제

내 답: _____ 확신도: 1 2 3 4 5

Q10. 익숙하지 않은 대규모 repo 에서 광범위한 리팩터링을 시작한다.

- A. 즉시 edit
- B. plan mode 로 탐색·계획 후 실행
- C. 모든 파일을 prompt 에 복사
- D. git history 삭제

내 답: _____ 확신도: 1 2 3 4 5

Q11. subagent 두 개가 동일 파일을 동시에 수정해야 한다.

- A. 항상 병렬이 빠르므로 병렬
- B. 충돌 가능성이 있으므로 순차/소유권 분리
- C. context window 만 키움
- D. tool description 변경

내 답: _____ 확신도: 1 2 3 4 5

Q12. 같은 긴 policy 문서 prefix 를 수천 번 반복 사용한다.

- A. 매번 그대로 전체 전송만
- B. prompt caching 고려
- C. 모델을 매번 변경
- D. subagent 10 개 추가

내 답: _____ 확신도: 1 2 3 4 5

Q13. agent 가 모호한 목표를 추정해서 20 분간 실행하면 재작업 비용이 매우 크다.

- A. 그냥 추정
- B. 핵심 ambiguity 를 시작 전에 clarification
- C. temperature 0
- D. history 삭제

내 답: _____ 확신도: 1 2 3 4 5

Q14. 팀 공통 프로젝트 규칙과 개인 로컬 실험 설정이 있다.

- A. 모두 user global 에 저장
- B. 공통은 project shared, 개인은 local 설정으로 분리
- C. 모두 enterprise policy
- D. 매번 prompt 에 직접 작성

내 답: _____ 확신도: 1 2 3 4 5

Q15. solver 가 infeasible 을 반환했다.

- A. 동일 입력 무한 retry
- B. infeasibility diagnostics branch 로 전환
- C. 무조건 feasible 로 답변 생성
- D. 오류를 숨기고 최종 답변

내 답: _____ 확신도: 1 2 3 4 5

10. 연습문제 정답과 해설

- Q1. B** - 독립 workstream 이므로 subagent 병렬화의 장점이 큼니다. coordinator 는 결과 통합만 담당합니다.
- Q2. B** - tool selection 문제는 먼저 이름·description·schema 를 개선하는 것이 가장 작은 해결책입니다.
- Q3. C** - 영향도가 큰 비가역 행동은 prompt-only 가 아니라 deterministic control/human approval 이 적절합니다.
- Q4. B** - 선행 결과가 다음 입력을 결정하므로 sequential dependency 입니다.
- Q5. B** - application boundary 에서는 typed/structured output 이 parsing 과 validation 을 안정화합니다.
- Q6. B** - 긴 context 는 요약·state 분리·선택적 context 로 관리합니다.
- Q7. B** - 조직 정책/보안은 모델의 준수 의지에 맡기지 않고 code/policy layer 에서 강제합니다.
- Q8. B** - least privilege 원칙. 필요한 read capability 만 노출합니다.
- Q9. B** - 오류 종류와 retry 가능 여부가 있어야 agent/application 이 다음 행동을 올바르게 결정합니다.
- Q10. B** - 큰 변경은 plan mode 로 먼저 탐색하고 전략을 검토하는 편이 안전합니다.
- Q11. B** - 동일 resource 동시 수정은 conflict 를 만들므로 serialization 또는 명확한 ownership 이 필요합니다.
- Q12. B** - 반복되는 큰 prefix 는 prompt caching 의 전형적인 적용 대상입니다.
- Q13. B** - 결과에 큰 영향을 주는 ambiguity 는 긴 autonomous run 전에 명확히 해야 합니다.
- Q14. B** - 공유 규칙과 개인 설정의 scope 를 분리해야 팀 일관성과 개인 유연성을 함께 얻습니다.
- Q15. B** - infeasible 은 동일 요청 반복보다 원인 분석을 위한 별도 recovery path 가 적절합니다.

11. 시험 직전 2 페이지 압축노트

Agentic Architecture

- single agent 가 기본값. multi-agent 는 isolation/parallelism/specialization 이점이 있을 때만.
- independent = parallel, dependency/shared mutation = sequential.
- hooks = deterministic enforcement / normalization / audit.
- resume = 이어가기, fork = 분기, checkpoint = 복구 지점.
- 비가역 행동은 최소 권한 + approval boundary.

Tool & MCP

- tool name/description/schema 가 routing 품질을 좌우.
- structured error: type + retryable + actionable detail.
- MCP 는 data/tool 연결 표준. protocol 자체보다 interface/permission 설계가 중요.
- tool surface 를 불필요하게 넓히지 말 것.

Claude Code

- CLAUDE.md = 프로젝트 작업 지침.
- 설정 우선순위: enterprise > CLI > local project > shared project > user.
- plan mode = 탐색/계획 후 실행.
- skill = 재사용 지식/절차, hook = 이벤트 기반 강제, subagent = 격리된 실행 주체.

Prompt / Structured Output

- clear & direct, success criteria 명시.
- few-shot 은 좋은 행동을 구체적으로 보여줌.
- machine consumption 이면 schema/structured output.
- 검증 실패는 targeted retry + limit.
- reviewer agent 는 오류 비용이 높을 때만.

Context & Reliability

- full history 를 무한 누적하지 말 것.
- summary / selective context / checkpoint / caching.
- ambiguity 의 영향도가 크면 먼저 clarification.
- timeout / partial failure / idempotency / provenance 를 설계에 포함.

최종 자기점검

- ☐ coordinator-subagent 구조가 필요한 이유를 30 초 안에 설명할 수 있다.
- ☐ parallel 과 sequential 을 의존성으로 구분할 수 있다.
- ☐ prompt instruction 과 hook enforcement 를 구분할 수 있다.
- ☐ 좋은 tool description/schema/error response 를 직접 작성할 수 있다.
- ☐ MCP 가 무엇을 표준화하는지 설명할 수 있다.
- ☐ CLAUDE.md/settings/skill/hook/subagent 차이를 말할 수 있다.
- ☐ structured output 을 써야 할 조건을 설명할 수 있다.
- ☐ 긴 context 의 문제와 해결책 3 가지를 말할 수 있다.
- ☐ least privilege 와 human approval boundary 를 설계에 적용할 수 있다.
- ☐ 모의문제를 120 분보다 빠르게 풀고, 선택지별 이유를 설명할 수 있다.

12. 10 회 학습 플랜

회차	주제	45~60 분 할 일	완료
1	시험 구조 + 판단 원칙	1~2 장 읽고 선택 공식 손으로 다시 쓰기	☐
2	Domain 1-1	agent loop / single vs multi / parallel vs sequential	☐
3	Domain 1-2	hooks / session / OptiQube Lab A	☐
4	Domain 2	tool schema / structured error / MCP + Lab B	☐
5	Domain 3	Claude Code config 를 실제 repo 에서 확인	☐
6	Domain 4	structured output + validation/retry 를 Python 으로 테스트	☐
7	Domain 5	context / caching / reliability + Lab C	☐
8	연습문제	15 문항 제한시간 30 분, 오답만 복습	☐
9	공식 문서 확인	약한 영역 2 개를 공식 docs 로 다시 읽기	☐
10	최종 점검	압축노트 2 페이지 + 외부 mock 1 회	☐

공부시간 추정

재형님처럼 Python/AI system 구현 경험이 있는 경우, 전체 내용을 처음부터 배우기보다 Claude 고유 개념과 시험식 판단을 보완하는 방향이 효율적입니다. 우선 8~12 시간을 1 차 목표로 잡고, 외부 mock 결과에 따라 약한 domain 만 추가 보강하는 방식을 권합니다.

13. 참고한 공식 자료 / 추가 학습 링크

Anthropic Partner Academy - Claude Certified Architect - Foundations

시험 형식, 도메인 비중, 가격/시간/유효기간

<https://anthropic-partners.skilljar.com/claude-certified-architect-foundations-certification>

Anthropic - Claude Partner Network

인증 프로그램 배경 및 partner certification

<https://www.anthropic.com/news/claude-partner-network>

Anthropic Docs - Tool use

tool 정의와 사용 패턴

<https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview>

Anthropic Docs - MCP

MCP 개요

<https://docs.anthropic.com/en/docs/mcp>

Anthropic Docs - Claude Code CLI

resume/continue/plan/automation 관련 명령

<https://docs.anthropic.com/en/docs/claude-code/cli-usage>

Anthropic Docs - Claude Code IAM/settings

설정 우선순위, permissions, hooks

<https://docs.anthropic.com/en/docs/claude-code/iam>

Anthropic Docs - Prompting best practices

명확한 지시, subagent 사용 판단 등

<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-templates-and-variables>

Anthropic Docs - Prompt engineering

출력 형식, 병렬 tool call 등

<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/claude-4-best-practices>

시험 blueprint 보조 확인

공개 Anthropic Academy 페이지에서 도메인 비중을 확인했고, task-level 세부 항목은 공개된 Exam Guide v1.0 을 요약한 독립 학습 사이트들과 교차 확인했습니다. 실제 시험 등록 전에는 Partner Academy 의 최신 Exam Guide 를 최종 기준으로 확인하세요.

마지막 메모**재형님에게 이 시험을 활용하는 방법**

CCAR-F 공부를 OptiQube 설계와 분리하지 마세요. 각 domain 을 공부할 때 OptiQube 에 “한 장짜리 설계 산출물”을 남기면, 자격증 공부와 그대로 업무/포트폴리오 자산이 됩니다.